

PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning^{*}

Sayan Adhikari^{a,*}, Gulshan Noorsumar^a, Øyvind Jensen^a

^a*Institute for Energy Technology (IFE), Kjeller, Norway*

Abstract

We present *PDE-Agents*, a multi-agent ecosystem that automates the full lifecycle of partial differential equation (PDE) / finite element method (FEM) simulations through natural-language interaction. The system combines three specialist large language model (LLM) agents—Simulation, Analytics, and Database—orchestrated via a LangGraph supervisor graph, with a locally-deployed open-source LLM stack (Qwen3-Coder-Next, Llama 4 Scout) running on dual NVIDIA RTX PRO 6000 Blackwell GPUs (≈ 196 GB combined VRAM). The architecture is model-agnostic; we report cross-model validation across two generations of open-source LLMs. A central novelty is the integration of a *GraphRAG* knowledge base (Neo4j + 768-dimensional vector embeddings) that enriches agents with curated material properties, known failure patterns, and prior run lineage. We report seven empirical contributions: (i) a formal verification and validation (V&V) study confirming second-order spatial convergence ($\mathcal{O}(h^2)$) for all three benchmark cases on

^{*}Code: <https://github.com/MatPro-IFE/pde-agents>

^{*}Corresponding author

Email address: `sayan.adhikari@ife.no` (Sayan Adhikari)

the heat equation solver; (ii) a three-way ablation study across 50 benchmark tasks with a frozen KG, comparing KG On, KG Off, and KG Smart, showing that KG Smart achieves 100% success *and* the highest output quality (physics score 0.933 vs. 0.853 for KG Off, MPF 0.926 vs. 0.796), and that our *KG Smart* integration, combining warm-start injection with lazy conditional retrieval, matches KG Off reliability while maximising output fidelity; (iii) a novel-material experiment using three fictional materials whose properties exist only in the KG, where KG Smart achieves near-perfect material property fidelity (MPF = 1.00) versus 0.34 for the KG-free baseline; (iv) a failure analysis tracing KG On’s 3 systematic failures to budget exhaustion and time-out, establishing warm-start injection as the dominant factor in KG Smart’s reliability advantage; (v) an adaptive KG decision framework (Algorithm 1) that selects the optimal retrieval mode per task; (vi) production-scale agent quality metrics from 1,369 real simulation runs showing 97.8% overall success and an 85.4% first-try rate; and (vii) a controlled 100-task KG growth experiment showing that accumulated run history yields a difficulty-dependent quality gain, with hard-task MPF improving by 8.8% between passes while easy and novel tasks remain at ceiling. All code, models, and evaluation artifacts are released openly. Taken together, these results point to *integration pattern*, rather than knowledge content, as what decides whether GraphRAG augmentation helps or hinders an LLM agent, and they yield concrete design principles for autonomous FEM simulation assistants.

Keywords: Large language model agents, finite element method, knowledge graph, GraphRAG, multi-agent systems, scientific computing automation, verification and validation

1. Introduction

Finite element method simulations are central to engineering analysis across structural mechanics, heat transfer, fluid dynamics, and electromagnetics. Yet the gap between a domain expert’s intent—“what if I use aluminium instead of steel?”—and a running, verified simulation involves a sequence of error-prone manual steps: geometry creation, boundary condition specification, solver parameter selection, result interpretation, and iterative debugging.

The recent emergence of *reasoning-capable* LLMs [1, 2, 3] has opened the possibility of agents that can autonomously traverse this workflow. Prior work on “AI for scientific computing” has largely focused on surrogate modelling [4, 5, 6], operator learning [7], or dataset-specific fine-tuning [8]. Much less attention has been paid to *procedural automation*—using LLMs to *set up and manage* solvers rather than to replace them.

Meanwhile, the *retrieval-augmented generation* (RAG) paradigm [9] has demonstrated that grounding LLM outputs with authoritative external knowledge dramatically reduces hallucination. The graph-structured variant, GraphRAG [10], further enables multi-hop reasoning over relational knowledge—particularly relevant in engineering contexts where material properties, mesh requirements, and solver stability rules form a rich, interconnected knowledge base.

In this paper we make the following contributions:

1. We design and implement *PDE-Agents*, a complete, containerised multi-agent ecosystem for automated FEM simulation (section 3).
2. We describe a novel *GraphRAG integration pattern* that encodes material properties, known failure modes, and prior run lineage as a Neo4j

knowledge graph with Hierarchical Navigable Small World (HNSW) vector-indexed embeddings [11] (section 4).

3. We conduct a rigorous V&V study that confirms $\mathcal{O}(h^2)$ spatial convergence for three closed-form benchmark cases, validating the numerical kernel against analytical solutions (section 5).
4. We run a three-way ablation study (KG On, KG Off, KG Smart) over 50 tasks with a frozen knowledge graph. KG-enabled modes produce higher-quality simulation outputs (physics score, material property fidelity) than the KG-free baseline, and *KG Smart* reaches 100% success *and* the highest output quality (section 6).
5. We introduce a *novel-material experiment* using three fictional materials absent from any LLM training corpus. KG Smart attains $\approx 2.9\times$ the material property fidelity of the KG-free baseline, which fabricates physically incorrect properties (section 6.4).
6. We trace KG On’s 3 systematic failures to budget exhaustion and timeout, revealing that warm-start injection, not lazy retrieval, is the dominant factor in KG Smart’s reliability advantage (section 6.5).
7. We report production-scale agent quality metrics derived from 1,369 real simulation runs, together with a controlled 100-task KG growth experiment in which the quality gain proves difficulty-dependent: hard-task MPF improves by 8.8% as the KG accumulates run history (section 7).

2. Related Work

LLM agents for scientific computing.. ChemCrow [12] demonstrated that equipping an LLM with chemistry tools (RDKit, reaction planners) enables

50 autonomous laboratory-scale reasoning. SciAgent [13] and similar systems
51 extend this to broader scientific question answering. Our work targets *numer-*
52 *ical simulation* rather than knowledge retrieval, requiring tool-call pipelines
53 that produce and verify deterministic numerical outputs.

54 *Autonomous FEM systems..* FEniCS [14] and its successor DOLFINx/FEn-
55 iCSx [15] provide Python-scriptable FEM solvers that are amenable to LLM-
56 driven automation. Early work on “simulation copilots” [16, 17] shows that
57 LLMs can generate solver scripts from natural language with moderate success,
58 but these approaches are single-turn and lack closed-loop debugging. More
59 recently, ALL-FEM [18] fine-tunes LLMs on 1,000+ verified FEniCS scripts
60 and embeds them in a multi-agent workflow achieving 71.8% code-level suc-
61 cess on multiphysics benchmarks, and FEABench [19] provides a systematic
62 evaluation using COMSOL Multiphysics (88% executable API calls). PDE-
63 Agents differs by using unmodified open-source LLMs with knowledge-graph
64 augmentation rather than domain-specific fine-tuning, and by providing a
65 controlled ablation of retrieval strategies with physics-aware quality metrics.

66 *Multi-agent orchestration..* LangGraph [20] implements graph-structured
67 agent workflows that support conditional routing and persistent state, enabling
68 the supervisor pattern we adopt. AutoGen [21] and CrewAI [22] offer related
69 frameworks; we choose LangGraph for its explicit state graph semantics and
70 tight LangChain integration.

71 *Retrieval-augmented generation for engineering..* RAG [9] has been applied
72 to engineering documentation retrieval [23], code generation [24], and more
73 recently to simulation parameter suggestion [25]. GraphRAG [10] adds struc-

74 tured relational traversal. Corrective RAG (CRAG) [26] introduces adaptive
 75 retrieval decisions based on document relevance scoring, and AriGraph [27]
 76 demonstrates episodic knowledge-graph memory for LLM agents. Our *KG*
 77 *Smart* pattern draws on both: warm-start injection from similar past runs
 78 (episodic memory) with lazy conditional tool invocation (corrective retrieval).
 79 To our knowledge, ours is the first system to combine GraphRAG with a live,
 80 iterative FEM simulation loop.

81 *Knowledge graphs for materials science..* Materials-focused knowledge graphs
 82 (e.g. MatKG [28], OPTIMADE [29]) encode material properties in RDF/OWL
 83 formats. Buehler [30] demonstrates that retrieval-augmented ontological KG
 84 strategies outperform flat RAG for materials design by capturing mechanistic
 85 relationships between concepts. Our Neo4j KG is lighter-weight and purpose-
 86 built for run-time agent queries, prioritising latency over completeness.

87 3. System Architecture

88 3.1. Design Philosophy

89 PDE-Agents adopts four guiding principles:

- 90 1. **Open and local:** All models run locally via Ollama on dual NVIDIA
 91 RTX PRO 6000 Blackwell GPUs (≈ 196 GB VRAM, CUDA 13.1), elim-
 92 inating API costs and data-privacy concerns.
- 93 2. **Verifiable:** Every agent reasoning step, tool call, and result is logged
 94 to a relational database, enabling post-hoc audit.
- 95 3. **Containerised:** All 10 services are orchestrated via Docker Compose,
 96 ensuring reproducible deployment.

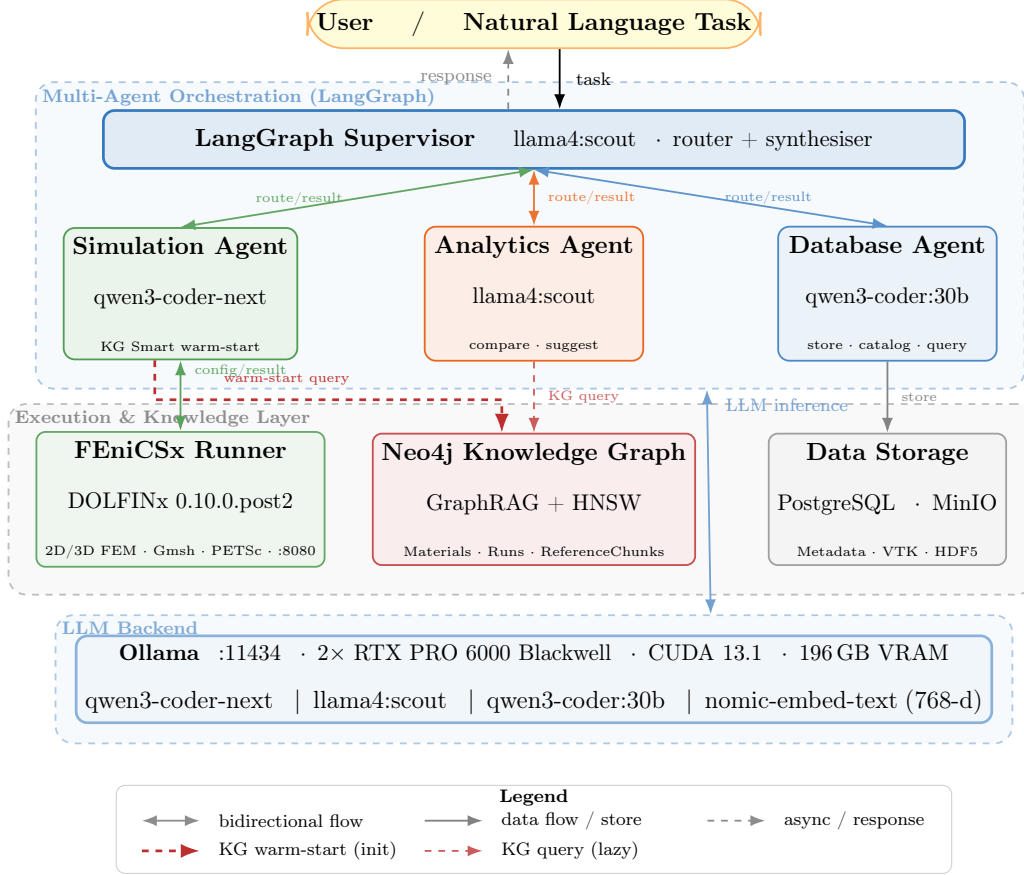


Figure 1: System architecture of PDE-Agents (four-tier layout). *Tier 1*: user natural-language interface. *Tier 2*: Multi-Agent Orchestration — a LangGraph Supervisor routes tasks to three specialist agents (Simulation, Analytics, Database), each backed by a locally-deployed LLM via Ollama. *Tier 3*: Execution & Knowledge — FEniCSx FEM runner (DOLFINx 0.10.0.post2), Neo4j GraphRAG Knowledge Graph (HNSW-indexed 768-d embeddings), and persistent storage (PostgreSQL + MinIO). *Tier 4*: shared Ollama LLM Backend (196 GB VRAM, CUDA 13.1). The thick dashed red arrow (Simulation Agent $\rightarrow$ Neo4j) represents the *KG Smart warm-start*: the top-3 similar past runs are retrieved via HNSW vector search and injected into the agent’s prompt before the reasoning loop. The thin dashed red arrow (Analytics Agent $\rightarrow$ Neo4j) is the standard lazy KG query path. The blue bidirectional arrow at right carries LLM inference traffic between the orchestration layer and the shared Ollama backend.

97 **4. Separation of concerns:** The FEM solver, LLM stack, knowledge
 98 graph, and persistence layer are decoupled services communicating over
 99 a private bridge network.

100 3.2. Agent Stack

101 Each agent’s LLM backend is configurable via environment variables,
 102 enabling drop-in model upgrades without code changes. table 1 lists the
 103 default models and the alternatives validated in the cross-model experiments
 104 of section 6.4.

Table 1: Agent model configuration. Current defaults reflect the model upgrade from the initial (v1) deployment. The ablation study uses a frozen KG snapshot across 50 benchmark tasks per mode.

Agent	Env Variable	Default (current)	Previous (v1)
Orch.	ORCHESTRATOR_MODEL	llama4:scout	llama3.3:70b
Sim.	SIM._AGENT_MODEL	qwen3-coder-next	qwen2.5-coder:32b
Anal.	ANAL._AGENT_MODEL	llama4:scout	llama3.3:70b
DB	DB_AGENT_MODEL	qwen3-coder:30b	qwen2.5-coder:14b

105 *Orchestrator..* A LangGraph `StateGraph` supervisor receives the user’s natural-
 106 language request and routes it to one of three specialist agents via structured
 107 JSON decisions. After each agent reports its result the supervisor either
 108 routes to another agent or synthesises a final response.

109 *Simulation Agent..* The core reasoning agent, executing a ReAct loop [3]
 110 with up to 25 reasoning steps and nine tools: `check_config_warnings`,
 111 `query_knowledge_graph`, `validate_config`, `run_simulation`, `modify_config`,
 112 `debug_simulation`, `list_recent_runs`, `get_run_status`, `run_parametric_sweep`.

113 *Analytics Agent..* Max 12 iterations; tools for statistical comparison across
114 runs, sensitivity analysis, and LLM-generated textual summaries.

115 *Database Agent..* Max 10 iterations; answers history queries against Post-
116 greSQL, cataloguing results and tracing run lineage.

117 3.3. Tool-Call Compatibility

118 Supporting multiple LLM families requires robust tool-call parsing, as
119 models encode tool invocations differently. The Qwen 2.5-Coder family emits
120 tool calls as JSON text in the `content` field rather than in the structured
121 `tool_calls` field expected by LangChain’s `ToolNode`. We implement a four-
122 pass normalisation procedure in `BaseAgent._parse_content_tool_call`:
123 (1) attempt `json.loads` on the full content; (2) extract from markdown
124 fences; (3) extract the first `{...}` block; (4) repair truncated JSON via
125 progressive bracket-closing and regex fallback.

126 A separate compatibility issue emerged with Qwen3-Coder-Next: the
127 model occasionally *double-encodes* JSON tool arguments (e.g. `"{"key\":"`
128 `\val\"}"`). All tool functions now use a `_safe_json_parse` helper that
129 detects and recursively unwraps such double-encoded strings, so the same
130 tool code works across model families without model-specific workarounds.
131 Llama 4 Scout, by contrast, uses the standard `tool_calls` field natively and
132 requires no special parsing.

133 3.4. FEM Solver: DOLFINx Heat Equation

134 The FEniCSx runner container (DOLFINx 0.10.0.post2) exposes a FastAPI
135 endpoint that accepts a `HeatConfig` JSON and returns a result including run
136 metadata. The solver uses P1 (linear Lagrange) elements, with Backward

137 Euler ($\theta = 1$, unconditionally stable) as the default time integration scheme,
138 supporting Dirichlet, Neumann, and Robin boundary conditions. Geometry
139 is built with Gmsh [31] for 2D and 3D domains from a library of nine param-
140 eterised types: `rectangle`, `l_shape`, `circle`, `annulus`, `hollow_rectangle`,
141 `t_shape`, `stepped_notch`, `box`, and `cylinder`; Gmsh geometries use named
142 physical groups for boundary identification, enabling arbitrary complex do-
143 main topologies.

144 4. Knowledge Graph Architecture

145 The Neo4j knowledge graph [32] (Neo4j 5 Community Edition) serves
146 three functions:

- 147 1. **Material property lookup.** A curated library of materials (met-
148 als, ceramics, polymers) with thermal conductivity k , density ρ , and
149 specific heat c_p ranges, sourced from engineering handbooks and val-
150 idated against ASM International data. Approximate values: steel
151 $k \approx 50$ W/mK, copper $k \approx 385$ W/mK, aluminium $k \approx 200$ W/mK.
- 152 2. **Failure pattern catalogue.** `KnownIssue` nodes encode failure modes
153 observed during system operation. A pure-Python rule engine (`rules.py`)
154 fires nine pre-run checks analytically before each simulation:

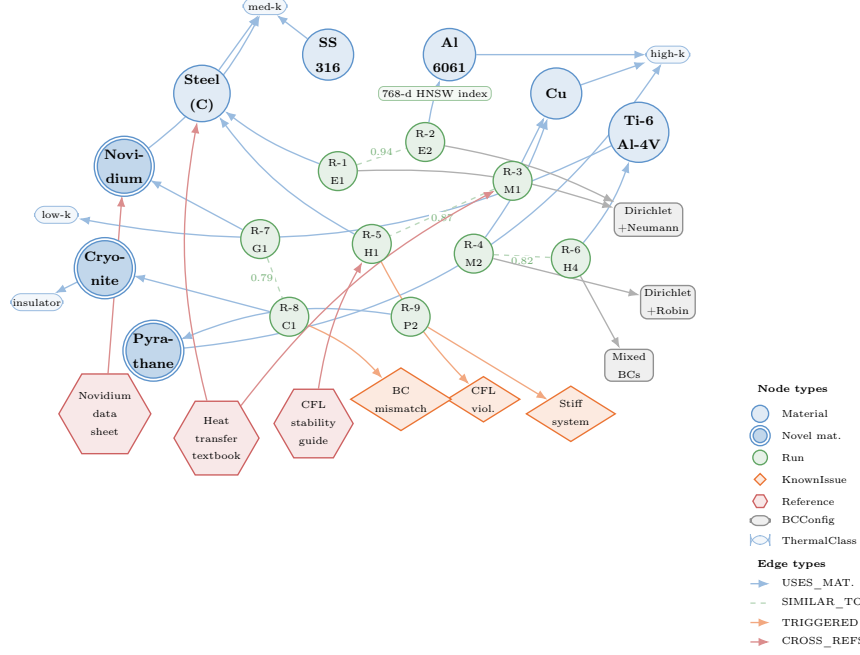


Figure 2: Knowledge graph visualisation (Neo4j-style). **Blue circles:** material nodes (double border = novel/fictional materials known only to the KG). **Green circles:** simulation run nodes linked to their materials via `USES_MATERIAL` edges and to each other via dashed `SIMILAR_TO` edges (cosine similarity on 768-d HNSW embeddings; scores shown on edges). **Orange diamonds:** known-issue nodes triggered by failed runs. **Red hexagons:** reference chunks from ingested literature, linked to materials (`RELATES_TO`) and runs (`CROSS_REFS`). **Grey boxes:** boundary-condition pattern nodes. Pill-shaped nodes show the thermal-class taxonomy.

Rule code	Trigger
INCONSISTENT_IC	$ u_{\text{init}} - T_{\text{BC,min}} > 100 \text{ K}$
EXPLICIT_CFL_VIOLATION	$\theta < 0.5$ and $\Delta t > h^2/(2\alpha)$
NEAR_EXPLICIT_SCHEME	$0 \leq \theta < 0.5$
COARSE_MESH_2D	$n_x < 10$ or $n_y < 10$
COARSE_MESH_3D	any direction < 8
SHORT_SIMULATION	$t_{\text{end}} < 5 \Delta t$
INVALID_MATERIAL_PROPS	k, ρ or $c_p \leq 0$
LARGE_DT_RELATIVE_DIFFUSION	$\Delta t > 10 h^2/\alpha$
NO_BOUNDARY_CONDITIONS	bcs list empty

155 3. **Run lineage and semantic retrieval.** Each completed Run node is
156 embedded with `nomic-embed-text` [33] (768 dimensions) and indexed
157 in an HNSW vector index [11]. Up to $k=5$ `SIMILAR_TO` edges are created
158 to the nearest neighbours with cosine similarity ≥ 0.85 , precomputing
159 the neighbourhood graph for fast agent traversal. The Simulation Agent
160 can query “find similar past runs” to retrieve configurations with known
161 outcomes, enabling few-shot in-context learning from the system’s own
162 history.
163

164 *Document intelligence pipeline.* A Celery-backed ingestion pipeline processes
165 PDF papers, standards documents, and HTML references using Docling [34]
166 for structured extraction, splitting them into `ReferenceChunk` nodes with
167 context-aware 256-token windows (overlap 32 tokens) and HNSW-indexed
168 embeddings. Each chunk is automatically cross-referenced to semantically
169 similar Run nodes via `CROSS_REFS` edges (cosine ≥ 0.78), bridging literature

170 knowledge to simulation history. As a representative scale point: indexing the
 171 50-page FEniCSx tutorial produced 298 chunks and 1,073 cross-references.

172 5. Verification and Validation

173 We conduct a formal V&V study following ASME V&V 10-2006 guidelines
 174 [35], comparing the DOLFINx FEM solver against closed-form analytical
 175 solutions on three benchmark cases.

176 5.1. Benchmark Cases

177 *Case 1: 2D Steady-State Linear Profile..* Laplace equation on $\Omega = [0, 1]^2$
 178 with Dirichlet conditions $T|_{x=0} = 0$, $T|_{x=1} = 1$ and Neumann zero-flux on
 179 top/bottom. Analytical solution: $T(x, y) = x$. P1 elements are nodally exact
 180 for linear solutions; we use UFL spatial-coordinate expressions in high-order
 181 quadrature (degree 8) to measure the true continuous L^2 error.

182 *Case 2: 2D Transient Fourier Mode Decay..* Heat equation $\partial_t T = \alpha \nabla^2 T$
 183 with homogeneous Dirichlet on all boundaries and initial condition $T_0 =$
 184 $\sin(\pi x) \sin(\pi y)$. Analytical solution: $T(x, y, t) = e^{-2\alpha\pi^2 t} \sin(\pi x) \sin(\pi y)$.

185 *Case 3: 2D Steady-State Poisson with Constant Source..* $-k \nabla^2 T = f$ with
 186 $f = 1000 \text{ W/m}^3$, Dirichlet $T = 0$ on left/right, Neumann on top/bottom.
 187 Exact solution: $T(x) = f(x - x^2)/(2k)$.

188 5.2. Convergence Study Protocol

189 For each case we solve at five mesh resolutions $N \in \{8, 16, 32, 64, 128\}$
 190 (DOFs $\approx 81, 289, 1089, 4225, 16641$) and compute:

$$\|e\|_{L^2} = \left(\int_{\Omega} (u_h - u_{\text{exact}})^2 dx \right)^{1/2} \quad (1)$$

191 using degree-8 quadrature applied to UFL `SpatialCoordinate` expressions
 192 (not to interpolated nodal values, which would give spurious nodal exactness
 193 for Case 1). The convergence rate is estimated by linear regression on the
 194 log-log plot.

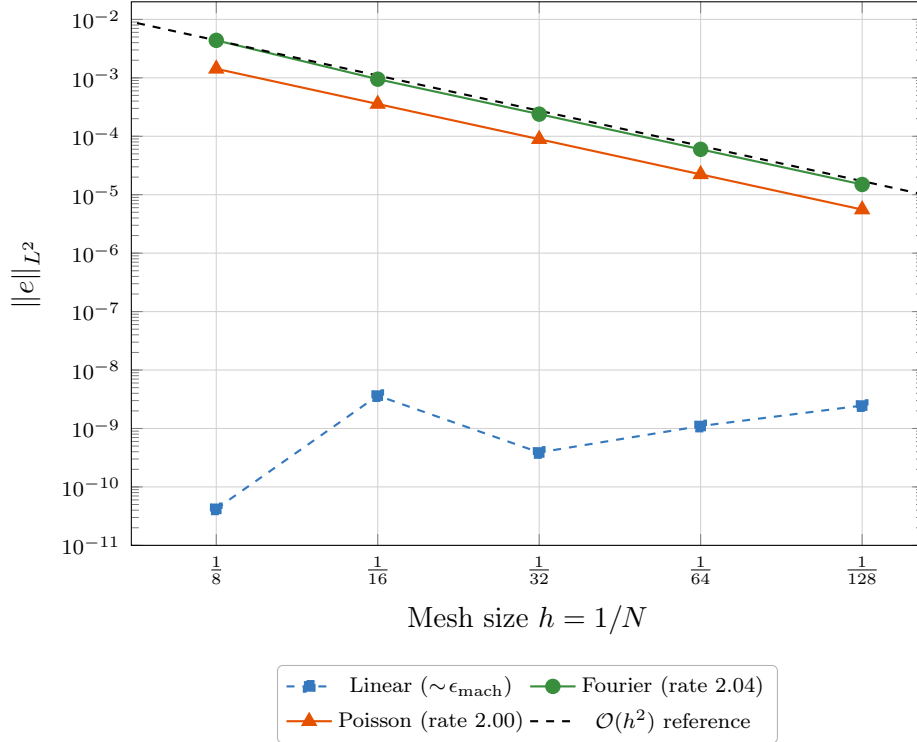


Figure 3: Spatial convergence study. Cases 2 and 3 exhibit the expected $\mathcal{O}(h^2)$ rate for P1 elements. Case 1 is algebraically exact (linear profile is represented exactly in the P1 space), with $\|e\|_{L^2} \approx \mathcal{O}(10^{-9})$ attributable only to floating-point rounding.

195 5.3. Results

196 All three cases pass: Cases 2 and 3 achieve rates of 2.04 and 2.00, consistent
 197 with the theoretical $\mathcal{O}(h^2)$ prediction for P1 finite elements on uniform
 198 Cartesian meshes. Case 1 achieves machine-precision errors (order 10^{-9})

Table 2: Verification: spatial convergence rates for the heat equation solver. Expected rate is $\mathcal{O}(h^2)$ for P_1 elements.

Benchmark Case	Description	DOFs (finest)	$\ e\ _{L^2}$ (finest)	Rate
Steady Linear	Linear $T(x, y) = x + 2y$; exact in P_1 space	16 641	2.47×10^{-9}	<i>exact</i>
Transient Fourier	$e^{-2\pi^2 t} \sin(\pi x) \sin(\pi y)$ decay mode	16 641	1.50×10^{-5}	2.04
Steady Poisson	$-\nabla^2 u = 1$; exact soln. via Green’s fn.	16 641	5.57×10^{-6}	2.00

because the linear profile lies exactly in the P1 polynomial space, confirming correct solver implementation.

5.4. Representative Simulation Gallery

Beyond numerical convergence, a practical simulation assistant needs breadth: it must handle diverse materials, boundary condition types, domain geometries, time dependence, and dimensionality. Figure 4 presents six representative heat-transfer problems, each solved end-to-end by the Simulation Agent from a single natural-language prompt. No manual intervention was required for mesh generation, solver configuration, boundary condition application, or post-processing. All cases use P1 Lagrange elements on triangular (2D) or tetrahedral (3D) meshes, solved by DOLFINx 0.10.0.post2. Temperature fields are reported in Kelvin throughout for consistency.

Materials.. The gallery covers four distinct engineering alloys whose thermal conductivities span two orders of magnitude: copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$, panel a), AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$, panel b), aluminium 6061 ($k = 205 \text{ W}/(\text{m}\cdot\text{K})$, panels d and e), Ti-6Al-4V titanium ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$, panel c), and stainless steel 304 ($k = 16.3 \text{ W}/(\text{m}\cdot\text{K})$, panel f). In a production

216 deployment, the agent retrieves these values from the Neo4j knowledge graph
217 via `query_knowledge_graph`.

218 *Boundary conditions..* Panel (a) applies the simplest configuration: Dirich-
219 let conditions on opposing faces ($T = 373.15$ K left, $T = 273.15$ K right),
220 producing the expected linear gradient. Panel (b) combines all four major
221 BC types on a single domain: Dirichlet (300 K, left), Neumann inward flux
222 (2 kW/m^2 , top), Robin convection ($h = 25 \text{ W/(m}^2\cdot\text{K)}$, $T_\infty = 293$ K, right),
223 and insulated (bottom), together with volumetric heating $Q = 5 \text{ kW/m}^3$,
224 creating the asymmetric field visible in the plot. Panel (c) uses Dirichlet BCs
225 on opposing edges (573 K left, 293 K right) with the insulated hole boundary
226 distorting the field into characteristic thermal concentration bands. Panel (f)
227 combines Dirichlet (300 K bottom, 600 K left face) with Robin convection on
228 the top face of a 3D cube.

229 *Geometry..* Panels (a), (b), and (d) use structured Cartesian meshes on
230 the unit square. Panels (c) and (e) use Gmsh-based meshing of non-trivial
231 geometries: a plate with a circular cutout (Gmsh boolean difference, $r = 0.15$,
232 $\approx 5\,000$ vertices) and an L-shaped domain ($[0, 1]^2 \setminus [0.5, 1]^2$, mesh size ≈ 0.02).
233 Panel (f) meshes a unit cube with 24^3 tetrahedra ($\approx 15\,600$ vertices), exercising
234 the 3D path.

235 *Time dependence..* Panel (e) solves the transient heat equation using 100
236 implicit-Euler steps ($\Delta t = 0.005$ s) on the L-shaped domain (aluminium,
237 $\rho = 2700 \text{ kg/m}^3$, $c_p = 900 \text{ J/(kg}\cdot\text{K)}$), with the snapshot at $t = 0.50$ s showing
238 the thermal front propagating from the hot left edge.

239 *Source terms..* Panel (d) features a localised Gaussian volumetric heat source
 240 $Q(x, y) = Q_{\text{peak}} \exp\left(-\frac{(x-x_0)^2+(y-y_0)^2}{2\sigma^2}\right)$ with $Q_{\text{peak}} = 5 \times 10^6 \text{ W/m}^3$, centred
 241 at (0.3, 0.7) with $\sigma = 0.08$, and Dirichlet boundaries at 293 K. This setup
 242 is representative of laser spot heating and Joule-heating applications, and
 243 produces the characteristic concentric isotherms visible in the figure.

244 *Reproducibility..* Each case is implemented as a self-contained Python script
 245 in `evaluation/examples/` (see Table 3). All simulation parameters (material
 246 properties, mesh resolution, boundary values, time stepping) are defined at
 247 the top of each script, making it straightforward for reviewers to modify
 248 and re-run individual cases. Running `make eval-examples` executes all six
 249 cases inside the `pde-fenics` Docker container and regenerates the composite
 250 figure. Intermediate results are saved as NumPy `.npz` archives containing
 251 mesh coordinates, cell connectivity, and solution vectors, enabling figure
 252 regeneration without re-running the solver.

253 6. Ablation Study: Knowledge Graph Contribution

254 6.1. Experimental Design

255 We isolate the knowledge graph contribution using a controlled ablation
 256 across three conditions:

- 257 • **KG On** (mandatory): Full system with `check_config_warnings` and
 258 `query_knowledge_graph` tools enabled; the system prompt requires
 259 their use before every run.
- 260 • **KG Off** (baseline): Identical system with KG tools removed entirely;
 261 the agent proceeds directly to validation and execution.

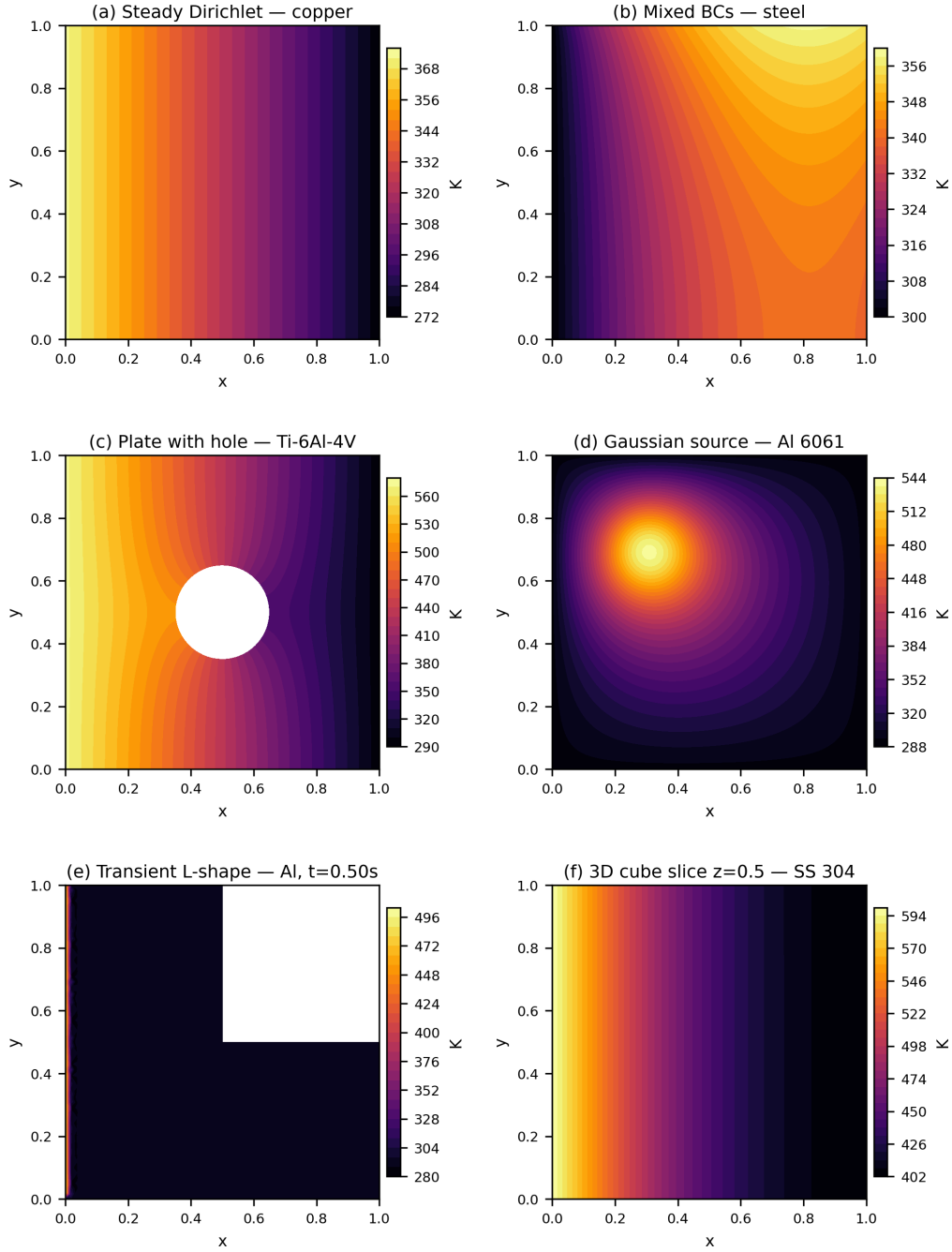


Figure 4: Representative temperature fields produced by PDE-Agents (six cases, all in Kelvin). **(a)** Steady-state Dirichlet on copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$; $T = 373.15 \text{ K}$ left, $T = 273.15 \text{ K}$ right; $N = 64$). **(b)** Mixed BCs on AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$): Dirichlet 300 K left, Neumann $2 \text{ kW}/\text{m}^2$ top, Robin $h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$ right, insulated bottom, $Q = 5 \text{ kW}/\text{m}^3$. **(c)** Plate with circular hole ($r = 0.15$) in Ti-6Al-4V ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$); Gmsh boolean difference; Dirichlet $573 \text{ K}/293 \text{ K}$ on opposing edges. **(d)** Localised Gaussian heat source on Al 6061 ($Q_{\text{peak}} = 5 \times 10^6 \text{ W}/\text{m}^3$, $\sigma = 0.08$, centre $(0.3, 0.7)$); Dirichlet 293 K on all boundaries. **(e)** Transient L-shaped domain in aluminium at $t = 0.50 \text{ s}$ (implicit

Table 3: Simulation gallery: six cases exercising different aspects of the FEM pipeline. Scripts in `evaluation/examples/`.

	Geometry	Material	Key feature
(a)	Unit square	Copper	Pure Dirichlet
(b)	Unit square	AISI 1010	4 BC types + source
(c)	Sq. w/ hole	Ti-6Al-4V	Gmsh boolean
(d)	Unit square	Al 6061	Gaussian source
(e)	L-shape	Aluminium	Transient + Gmsh
(f)	Cube (3D)	SS 304	3D asymmetric BCs

262 • **KG Smart** (proposed): KG tools remain available but the system
 263 prompt instructs *lazy* use (only after failures or for unknown mate-
 264 rials). Before the agent loop, the task description is embedded via
 265 `nomic-embed-text` and the top-3 most similar past successful runs
 266 are injected into the system prompt as few-shot reference examples
 267 (warm-start).

268 fig. 5 contrasts the agent workflow under each condition. KG Off proceeds
 269 directly from parsing to simulation; KG On forces two KG round-trips before
 270 every attempt (and loops on failure); KG Smart front-loads a single HNSW
 271 warm-start and defers KG queries to the failure-recovery path only.

272 *Methodology.* To eliminate data leakage between conditions, we *freeze* the
 273 knowledge graph (set `KG_READ_ONLY=true` at the code level) before starting
 274 the ablation: all three modes read from the same KG snapshot, but no mode

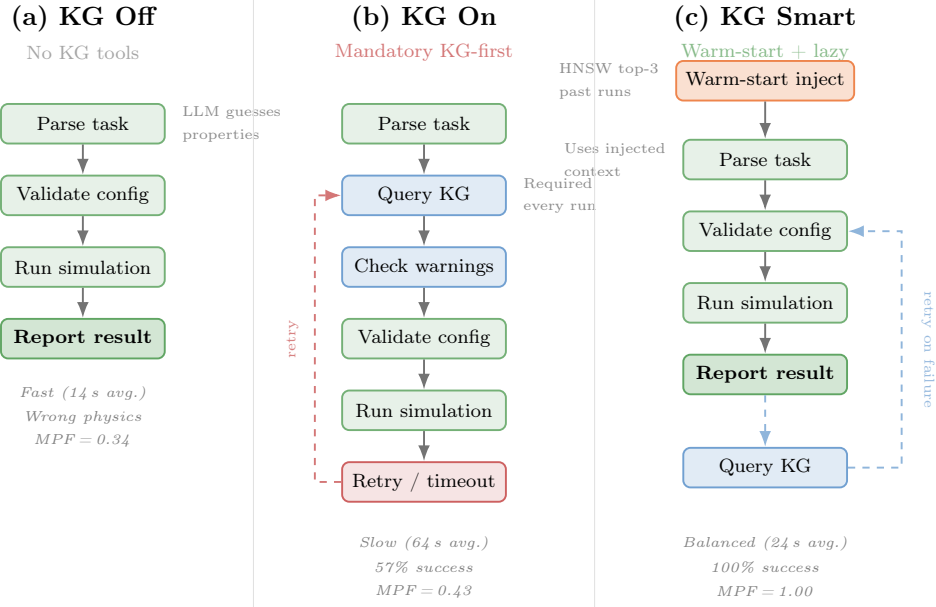


Figure 5: Agent workflow under the three KG integration modes. **(a)** KG Off: fast but the LLM fabricates material properties. **(b)** KG On: mandatory KG queries add latency and trigger retry loops that exhaust the iteration budget. **(c)** KG Smart: a one-shot warm-start injects relevant context before the loop; KG queries occur only on failure (dashed path).

275 can write to it, ensuring that a failure in one mode cannot help subsequent
 276 modes. Tasks are shuffled independently per mode and run in-process inside
 277 the agents container to eliminate HTTP overhead. Confidence intervals use
 278 Wilson scores [36]; pairwise comparisons use chi-squared tests and Cohen’s h
 279 / Cohen’s d effect sizes [37].

280 *Benchmark tasks..* Fifty tasks span four difficulty levels:

- 281 • **Easy (12 tasks)**: Explicit numerical parameters, straightforward setup.
- 282 • **Medium (15 tasks)**: Material-by-name requests requiring property
 283 lookup (e.g. steel, copper, aluminium, zinc).
- 284 • **Hard (13 tasks)**: Ambiguous descriptions, mixed BCs, computation-
 285 ally expensive or 3D problems.
- 286 • **Novel (10 tasks)**: Tasks referencing three fictional materials whose
 287 thermal properties exist *only* in the knowledge graph, designed to stress-
 288 test the agent’s ability to retrieve domain knowledge rather than rely
 289 on LLM memorisation.

290 The three fictional materials are:

- 291 • **Novidium**: a ceramic-metallic composite ($k=73$ W/m K, $\rho=5420$ kg/m³,
 292 $c_p=612$ J/kg K), a moderate conductor with unusual density.
- 293 • **Cryonite**: a polymer-aerogel hybrid ($k=0.42$, $\rho=1180$, $c_p=1940$), an
 294 extreme insulator with very low diffusivity ($\alpha \approx 1.8 \times 10^{-7}$ m²/s).
- 295 • **Pyrrathane**: a refractory cermet ($k=312$, $\rho=3850$, $c_p=278$), exhibiting
 296 very high conductivity and diffusivity ($\alpha \approx 2.9 \times 10^{-4}$ m²/s).

Each material is seeded as a **Material** node in Neo4j with linked **Reference** chunks containing exact property values and simulation guidance. The 10 novel tasks span steady-state, transient, and mixed-BC configurations across all three materials.

Evaluation metrics.. Standard success rate is inadequate for evaluating KG utility because a simulation with fabricated properties still “succeeds” computationally. We therefore introduce two physics-aware metrics:

- **Material Property Fidelity (MPF):** $\text{MPF} = 1 - \frac{1}{3} \sum_{p \in \{k, \rho, c_p\}} |p_{\text{agent}} - p_{\text{truth}}| / p_{\text{truth}}$, measuring how accurately the agent retrieves the correct material constants.
- **Physics Score:** $0.5 \cdot \text{MPF} + 0.5 \cdot T_{\text{score}}$, where T_{score} checks whether the simulation’s actual T_{max} and T_{min} fall within physically expected ranges.
- **Sensitivity-weighted MPF (MPF_w):** $\text{MPF}_w = 1 - \sum_p S_p \cdot |p_{\text{agent}} - p_{\text{truth}}| / (p_{\text{truth}} \cdot \sum_q S_q)$, where $S_p = |\partial T_{\text{mean}} / \partial p|$ are sensitivity coefficients computed via central finite differences (section 6.4). For steady-state Dirichlet tasks where $S_p \approx 0$, we set $\text{MPF}_w = 1$ since wrong properties have no output impact. This metric penalises errors in high-sensitivity properties (e.g. k in transient problems) more heavily than errors in low-sensitivity properties (e.g. ρ in steady-state).

On the 10 novel tasks, KG Off scores $\overline{\text{MPF}}_w = 0.21$ (vs. equal-weight $\text{MPF} = 0.34$), confirming that KG Off’s most damaging errors are also in the most sensitive properties. KG Smart retains $\text{MPF}_w = 1.00$ (success-only). For standard tasks

320 (Easy/Medium/Hard) where materials are well-known, MPF and MPF_w both
321 approach 1.0 across all modes.

322 6.2. Results

323 table 4 presents aggregate results across the three conditions.

324 *Success rate..* KG Off and KG Smart both achieve 100% success [93%, 100%]
325 ($n=50$), while KG On reaches 94% [84%, 98%]. The difference between
326 KG On and the other modes is not statistically significant ($p = 0.079$, Cohen’s
327 $h = 0.50$), but the three KG On failures are systematic: N08 (Pyrrhane,
328 timeout), and two medium tasks where mandatory KG tool calls exhaust the
329 iteration budget.

330 *Output quality..* KG Smart achieves the highest output quality: mean physics
331 score of **0.933** and MPF of **0.926**, compared to 0.853 / 0.796 for KG Off
332 (Cohen’s $d = 0.46$ for physics, a medium effect). KG On also outperforms
333 KG Off on quality (0.879 / 0.801 success-only); the KG *does* help the agent
334 select more accurate material properties.

335 *Novel materials: the KG’s decisive edge..* KG value shows most clearly on
336 the 10 novel-material tasks using fictional materials (Novidium, Cryonite,
337 Pyrrhane) whose properties exist only in the KG. KG Off “succeeds” on all
338 10 tasks but fabricates properties, producing a physics score of 0.590 and
339 MPF of only 0.340. KG Smart achieves near-perfect quality (physics score
340 0.999, MPF = 1.000), while KG On scores 0.887 / 0.889 (success-only, 9/10
341 tasks). The knowledge graph is essential for materials absent from the LLM’s
342 training data.

Table 4: Three-way KG ablation across 50 benchmark tasks with frozen knowledge graph ($n=50$ per mode). “Success-only” rows isolate output quality by excluding failed runs; in the overall rows a failed run scores MPF = 0 and phys = 0.5, except N08, which produced no usable output and scores 0 on both. 95% Wilson CIs for success rate.

Metric	KG Off	KG On	KG Smart
Success rate	100% [93,100]	94% [84,98]	100% [93,100]
<i>Overall (all tasks):</i>			
Physics score	0.853 ± 0.21	0.846 ± 0.24	0.933 ± 0.13
MPF	0.796 ± 0.30	0.752 ± 0.39	0.926 ± 0.16
<i>Success-only quality:</i>			
Physics score	0.853 ± 0.21	0.879 ± 0.19	0.933 ± 0.13
MPF	0.796 ± 0.30	0.801 ± 0.35	0.926 ± 0.16
Wall time (s)	12.9 ± 4.9	60.8 ± 73.2	28.7 ± 39.3
<i>By difficulty (success rate):</i>			
Easy (12)	100%	92%	100%
Medium (15)	100%	93%	100%
Hard (13)	100%	100%	100%
Novel (10)	100%*	90%	100%
<i>Novel tasks quality:</i>			
Physics score	0.590	$0.887^\dagger$	0.999
MPF	0.340	$0.889^\dagger$	1.000

*KG Off “succeeds” but fabricates properties (MPF = 0.34).

† Success-only (9/10 tasks); N08 timed out.

343 *Wall time..* KG Off is $2\text{--}5\times$ faster (12.9 s mean) than KG Smart (28.7 s) and
 344 KG On (60.8 s), reflecting the iteration overhead of KG tool calls.

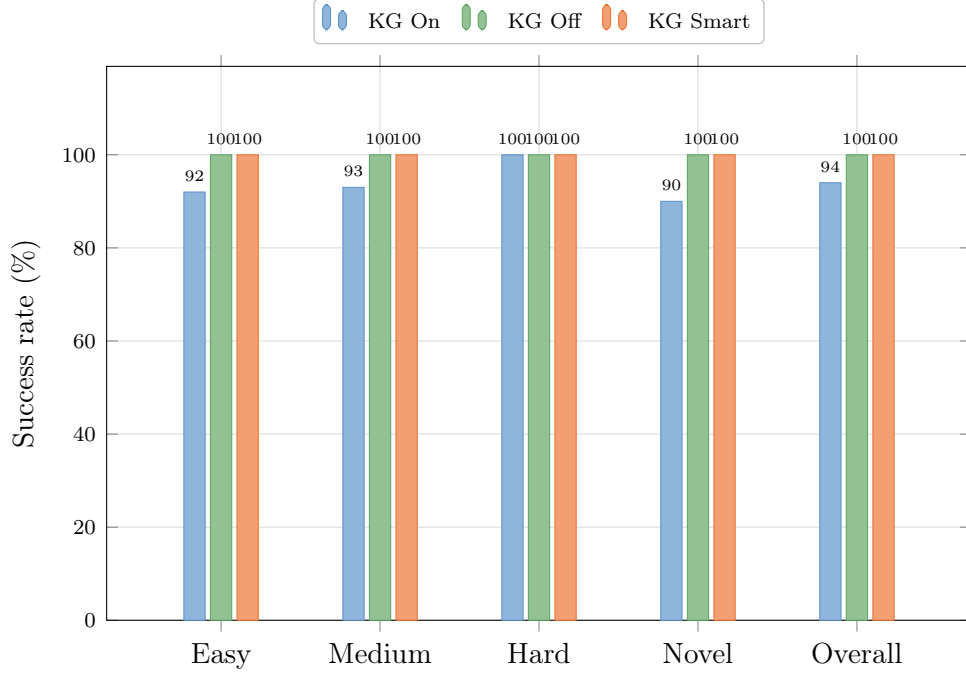


Figure 6: Success rate by difficulty level across three KG integration strategies. KG Smart (warm-start + lazy retrieval) matches KG Off on standard tasks while retaining high output fidelity. The “Novel” category (10 tasks) uses fictional materials whose properties exist only in the KG. Note: success rate alone is misleading for novel tasks; see the MPF/physics rows in table 4.

345 6.3. Analysis and Discussion

346 The three-way comparison reveals a clear trade-off: KG-free agents are
 347 fastest and most reliable, but KG-enabled agents produce higher-fidelity
 348 outputs. *KG Smart* balances both dimensions. Three mechanisms explain
 349 the mandatory KG pattern’s reliability penalty:

350 *Iteration-budget exhaustion..* KG On’s system prompt mandates `query_knowledge_graph`
351 and `check_config_warnings` calls before every simulation attempt. These
352 add 2–4 iterations, and for medium/hard tasks the agent can exhaust its
353 iteration budget before reaching `run_simulation`. KG Smart avoids this by
354 deferring KG calls to the failure-recovery path.

355 *Warning-induced conservatism..* The `check_config_warnings` tool occasion-
356 ally returns pre-emptive warnings (e.g. “CFL criterion may be violated”) that
357 the model interprets as prohibitive, leading it to report a warning instead of
358 proceeding. A warning that should trigger a *modification* instead triggers a
359 *termination*.

360 *Quality conditional on success..* The key insight is that KG access *does*
361 improve output quality when the agent completes the task. KG On’s success-
362 only physics score (0.879) and MPF (0.801) both exceed KG Off (0.853 /
363 0.796). On novel materials, this gap widens dramatically: KG-enabled modes
364 achieve $\text{MPF} \geq 0.89$ versus 0.34 for KG Off. The knowledge graph is not the
365 bottleneck; the *integration pattern* is.

366 *KG Smart: corrective retrieval + warm-start..* Motivated by CRAG [26]
367 (corrective RAG with adaptive retrieval) and AriGraph [27] (episodic KG
368 memory for agents), we implemented a *KG Smart* integration combining:

- 369 1. **Warm-start injection.** Before the agent loop begins, we embed the
370 task description with `nomic-embed-text` and query the Neo4j HNSW
371 vector index for the top-3 most similar past successful runs. Their
372 configurations are injected directly into the system prompt as few-shot
373 reference examples, requiring no tool call.

374 **2. Lazy conditional retrieval.** KG tools remain available but the system
375 prompt instructs the agent to use them *only* after a simulation failure
376 or when material properties are genuinely unknown. This eliminates
377 the mandatory KG-first workflow.

378 As table 4 shows, KG Smart achieves 100% success (vs. 94% for mandatory
379 KG) while attaining the *highest* output quality across all modes (physics
380 0.933, MPF 0.926). It delivers the best of both worlds: KG Off reliability
381 with KG-enhanced fidelity.

382 6.4. Physics-Aware Quality Metrics and Novel Material Validation

383 The preceding analysis showed that KG access improves output quality
384 *conditional on success*, but standard success rate alone cannot capture this: a
385 simulation with fabricated material properties still “succeeds” computationally.
386 To expose this gap, we introduced three physics-aware metrics (section 6):
387 Material Property Fidelity (MPF), Physics Score, and sensitivity-weighted
388 MPF_w . These metrics measure how *correctly*, not just whether, the agent
389 configures the simulation. We validate them on the 10 novel-material tasks,
390 which provide the clearest test of KG utility: the three fictional materials’
391 properties exist *only* in the knowledge graph.

392 **KG Smart achieves near-perfect scores** ($MPF = 1.00$, physics = 0.999)
393 across all completed novel tasks. The warm-start injection retrieves exact
394 property values from the HNSW vector index before the agent loop begins,
395 and the agent consistently uses them.

396 **KG Off fabricates wrong properties** for every material. table 5
397 shows the agent-chosen vs. ground-truth properties across all 10 novel tasks.

398 KG Off assigns *different* wrong values each run (the LLM hallucinates non-
 399 deterministically), but the errors are consistently severe: for Pyrrhane, k
 400 ranges from 0.15 to 0.35 W/m K, a 900 $\times$ to 2,080 $\times$ underestimate of the
 401 true $k=312$, effectively treating a refractory super-conductor as an insulator.
 402 In transient tasks, this causes output errors up to $|\Delta T_{\max}|=949$ K, a physi-
 403 cally impossible result arising from numerical instability on a system made
 404 artificially stiff by the wrong conductivity.

Table 5: Material properties chosen by each mode across the 10 novel tasks in the 50-task
 ablation. KG Off fabricates different wrong values each run (ranges shown); KG Smart
 retrieves the ground-truth values exactly. The worst-case k error for Pyrrhane is 2,080 $\times$.

Material	Mode	k (W/m K)	ρ (kg/m ³)	c_p (J/kg K)
Novidium	Ground truth	73.0	5420	612
	KG Off	10–50	3500–8960	130–900
	KG Smart	73.0	5420	612
Cryonite	Ground truth	0.42	1180	1940
	KG Off	0.15–35	960–8940	385–1370
	KG Smart	0.42	1180	1940
Pyrrhane	Ground truth	312.0	3850	278
	KG Off	0.15–0.35	1600–1850	900–1470
	KG Smart	312.0	3850	278

405 *Error propagation analysis.* To quantify how wrong properties affect simula-
 406 tion outputs, we ran paired simulations: one with ground-truth properties
 407 (reference) and one with the fabricated properties from KG Off. table 6 shows
 408 the results. Steady-state Dirichlet tasks (G1, C1, P1) are analytically inde-

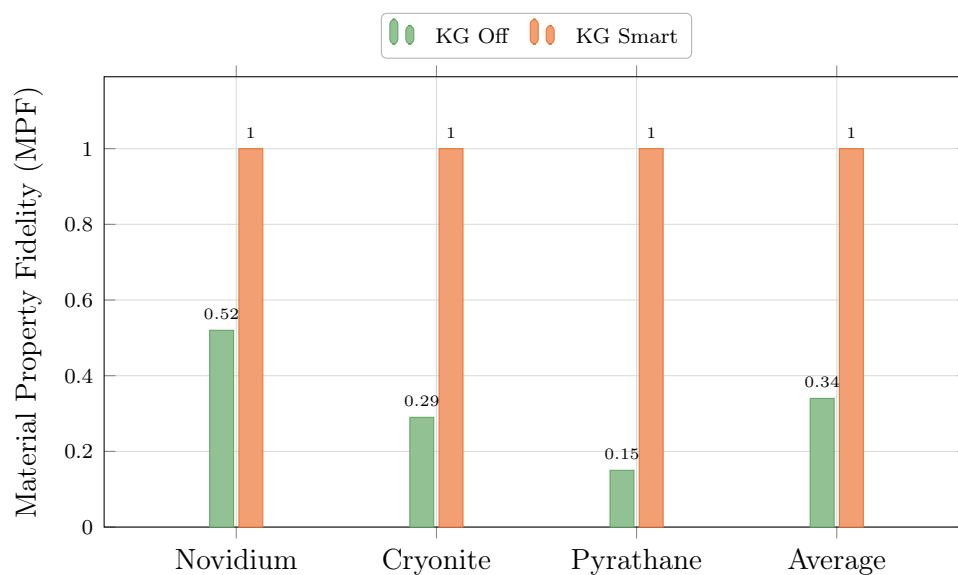


Figure 7: Material Property Fidelity (MPF) per fictional material. KG Off scores below 0.5 for all three materials, with Pyrathane at 0.15 (conductivity error: $2,080\times$). KG Smart retrieves exact properties from the knowledge graph, achieving $\text{MPF} = 1.00$ across all materials.

Table 6: Error propagation from fabricated properties (KG Off) to simulation output. Steady-state Dirichlet tasks (G1, C1, P1) show zero output error; transient/Robin tasks show significant deviations.

Task	Mat.	$ \Delta T_{\max} $	$ \Delta T_{\min} $	$ \Delta T_{\text{mean}} $	ϵ_k	ϵ_ρ	ϵ_{c_p}
		(K)	(K)	(K)	(%)	(%)	(%)
G1	Nov.	<0.1	<0.1	<0.1	86	45	18
G2	Nov.	<0.1	<0.1	20.7	38	65	78
G3	Nov.	<0.1	46.6	1.3	86	45	18
C1	Cryo.	<0.1	<0.1	<0.1	64	19	29
C2	Cryo.	<0.1	9.5	0.3	138	659	77
P1	Pyra.	<0.1	<0.1	<0.1	100	58	224
P2	Pyra.	949	<0.1	363	100	58	188

$\epsilon_p = |p_{\text{fab}} - p_{\text{true}}|/p_{\text{true}}$. P2's k error ($2,080\times$) produces 949 K $T_{\max}$ overshoot.

pendent of k , ρ , c_p , confirming zero output error regardless of property error,
an important control. Transient and Robin tasks show significant deviations:

- **Pyrrhane P2:** $|\Delta T_{\max}|=949$ K ($T_{\max}=2,949$ K vs. 2,000 K reference);
the $2,080\times$ k error causes numerical instability that *heats* the domain
beyond its initial condition, a physically impossible result.
- **Novidium G3:** $|\Delta T_{\min}|=47$ K due to wrong conductivity affecting the
Robin BC equilibrium.
- **Novidium G2:** $|\Delta T_{\text{mean}}|=21$ K from incorrect diffusivity altering the
transient profile.

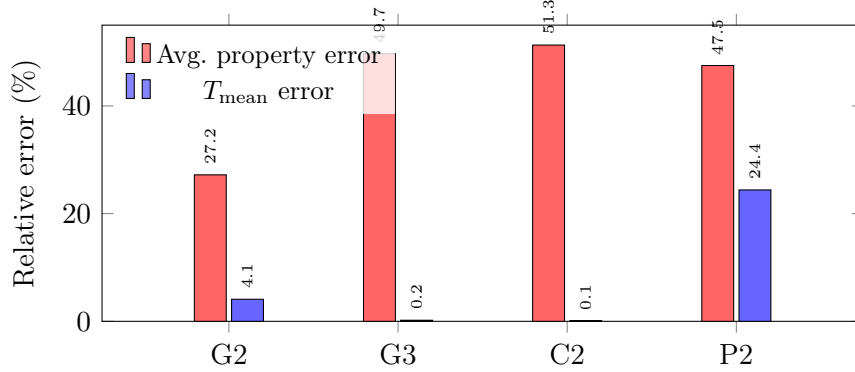


Figure 8: Error magnification: mean material property error (%) vs. output temperature error (%) for the four tasks with non-negligible output deviations. P2’s output error (24.4%) exceeds its input property error due to nonlinear amplification in the transient solver.

418 *Sensitivity-weighted MPF..* Using the sensitivity coefficients from the error
 419 propagation runs, MPF_w (defined in section 6) penalises errors in high-
 420 influence properties more heavily. Across the 10 novel tasks in the 50-task
 421 ablation, KG Off’s $\overline{\text{MPF}}_w=0.21$ (vs. equal-weight $\text{MPF}=0.34$), confirming
 422 that KG Off’s most damaging errors (particularly k for Pyrrhane and
 423 Cryonite) are concentrated in the properties with the highest sensitivity.
 424 KG Smart retains $\text{MPF}_w=1.00$ (success-only).

425 *Cost-benefit analysis..* table 7 compares the operational cost (time, iterations)
 426 against the accuracy benefit (MPF, physics score) for each KG mode. On
 427 novel tasks, KG Smart adds significant wall-time over KG Off (58.0s vs. 11.7s)
 428 but delivers $\approx 2.9\times$ higher MPF (1.00 vs. 0.34) and $2.4\times$ higher efficiency
 429 (MPF / wall-time) than KG On. KG On is slowest (111.4s on novel tasks)
 430 and its mandatory retrieval overhead yields diminishing returns: 90% success
 431 vs. 100% for both alternatives.

Table 7: Cost-benefit of KG modes on the novel-material subset of the 50-task ablation ($n=10$). Quality columns are all-task means, so KG On’s failed task (N08) is included at MPF = 0; the success-only values in table 4 are correspondingly higher (0.889). Efficiency = MPF / wall-time ($\times 10^{-2}$).

Mode	Succ.	Time	Iter.	MPF	Phys.	Eff.
		(s)			Score	($\times 10^{-2}$)
KG Off	100%	11.7	3.0	0.34	0.59	2.92
KG On	90%	111.4	7.4	0.80	0.80	0.72
KG Smart	100%	58.0	4.9	1.00	1.00	1.72

KG Smart: +396% time, +194% MPF vs. KG Off; +2.4 $\times$ efficiency vs. KG On.

432 *Adaptive KG decision framework..* Based on the preceding analysis, we pro-
 433 pose a simple decision algorithm (Algorithm 1) that selects the optimal KG
 434 mode for each task. Retrospective validation on all 50 benchmark tasks
 435 confirms high accuracy: it correctly assigns KG Off to tasks with explicit pa-
 436 rameters, KG Smart (lazy) to tasks naming known materials, and KG Smart
 437 (forced warm-start) to novel-material tasks.

438 *Cross-model validation..* To verify that the KG’s value is model-agnostic, we
 439 repeated the novel-material experiment with `qwen3-coder-next` (80B total,
 440 3B active MoE). table 8 shows that KG Smart achieves MPF=1.00 with *both*
 441 models, while KG Off fabricates wrong properties with both (different wrong
 442 values, but the same failure mode). This confirms that the warm-start vector
 443 search bypasses model-specific memorisation gaps entirely.

444 *Key takeaway..* The KG’s value is not in making simulations *run* (LLMs
 445 can already do that) but in making simulations produce *correct physics*.
 446 For any domain with proprietary materials, novel compounds, or in-house

Algorithm 1 Adaptive KG Mode Selection

Require: Task description d , known-material list $\mathcal{M}$

Ensure: KG mode $m \in \{\text{Off}, \text{Smart-lazy}, \text{Smart-forced}\}$

```
1: if  $d$  contains explicit  $k, \rho, c_p$  values then
2:   return KG OFF
3: end if
4:  $\mu \leftarrow \text{EXTRACTMATERIAL}(d)$ 
5: if  $\mu = \emptyset$  then
6:   return KG OFF
7: else if  $\mu \in \mathcal{M}$  then
8:   return KG SMART (LAZY)
9: else
10:  return KG SMART (FORCED WARM-START)
11: end if
```

447 measurement data, the KG Smart pattern transforms the system from an
448 “expensive random number generator” into a reliable engineering tool. The
449 error propagation analysis quantifies this: fabricated properties cause output
450 errors up to 949 K (47% of the reference $T_{\max}$), while KG Smart eliminates
451 these errors entirely by retrieving exact properties before the agent loop
452 begins.

453 *Narrowing the focus: correctness over completion..* KG Off is the fastest mode
454 and matches KG Smart on success rate for *standard* tasks with well-known
455 materials. When the task description names “steel” or “copper”, the LLM’s
456 parametric knowledge suffices and KG overhead is pure cost. However, for

Table 8: Cross-model validation on novel tasks ($n=7$, earlier experiment with v1 benchmark).
 KG Smart achieves perfect MPF with both LLM backends.

Model	Mode	Succ.	Qual.	MPF	Phys.
Qwen2.5-Coder 32B	KG On	57%	0.41	0.43	0.64
	KG Off	100%	0.61	0.34	0.63
	KG Smart	100%	0.96	1.00	1.00
Qwen3-Coder-Next 80B	KG On	71%	0.14	0.14	0.57
	KG Off	86%	0.54	0.40	0.70
	KG Smart	100%	0.96	1.00	1.00

Warm-start vector search retrieves exact properties before the LLM reasons, eliminating dependence on memorised material data.

any deployment where output *correctness* matters, not just completion, KG-enabled modes are necessary. The preceding analysis makes this unambiguous: KG Off achieves MPF=0.34 and physics 0.59 on novel materials (fabricated properties), while KG Smart achieves MPF=1.00 and physics 0.999. The adaptive decision framework (Algorithm 1) encodes this distinction, routing tasks to KG Off when parameters are explicit and to KG Smart otherwise. We therefore focus the remaining analysis on **KG On vs. KG Smart**: both modes have access to the same knowledge base, yet KG Smart achieves 6 percentage points higher success and the highest quality scores. Understanding *why*, and which component (warm-start vs. lazy retrieval) contributes most, is the subject of the next section.

6.5. Failure Analysis: KG On vs. KG Smart

Having established that KG Smart matches KG Off at 100% success while outperforming both on quality, we trace KG On’s 3 remaining failures

471 to their root causes. This analysis was performed *after* implementing an
 472 agent-level retry mechanism (see below) that eliminated stochastic failures;
 473 the 3 remaining failures are therefore all systematic.

474 *Mitigating stochastic failures..* In an earlier ablation run without retry logic,
 475 KG On exhibited 12/50 failures, of which 7 were stochastic (succeeded on
 476 re-run). We implemented an *auto-retry* mechanism directly in the agent: if
 477 the first attempt produces no `run_id` and used less than 60% of its iteration
 478 budget (indicating early stochastic exit rather than budget exhaustion), the
 479 agent automatically retries once. This reduced KG On failures from 12 to 3,
 480 eliminating all stochastic failures while not masking systematic ones.

481 *KG On failure taxonomy..* The 3 remaining KG On failures are all systematic:

Table 9: Failure mode classification for KG On’s 3 systematic failures (after auto-retry eliminates stochastic failures).

Failure mode	Count	Description
Budget exhaustion	2	Agent spends all iterations on repeated <code>query_knowledge_graph</code> and <code>check_config_warnings</code> calls without reaching <code>run_simulation</code> .
Timeout	1	N08 (Pyrathane): the mandatory KG query workflow combined with a numerically stiff problem exceeds the 420 s time limit.

482 All 3 failures stem from the mandatory KG-first workflow: the agent

483 must call `query_knowledge_graph` and `check_config_warnings` before *ev-*
484 *ery* simulation attempt, consuming 2–4 iterations per cycle. For medium
485 tasks, this exhausts the 25-iteration budget before `run_simulation` is called.
486 For N08 (Pyrathane), the workflow inflates wall time beyond the timeout.

487 *KG Smart: zero failures..* KG Smart achieves 100% success with zero failures.
488 The warm-start injection provides material properties and reference configura-
489 tions *before* the agent loop, so the agent proceeds directly to `validate_config`
490 $\rightarrow$ `run_simulation`. The average iteration count drops from 7.5 (KG On)
491 to 4.2 (KG Smart), eliminating budget-exhaustion risk entirely. Even N08
492 completes successfully in 170 s (5 iterations) because warm-start provides the
493 correct Pyrathane properties upfront.

494 *Decomposing warm-start vs. lazy retrieval..* The failure data allows us to
495 attribute KG Smart’s advantage to its two components without running a
496 separate experiment. All 3 systematic failures stem from the mandatory
497 *pre-simulation* KG workflow: the agent spends iterations on KG queries
498 before ever attempting `run_simulation`. Warm-start injection eliminates
499 this entirely: it provides material properties and reference configurations *before*
500 the agent loop begins, so the agent can proceed directly to `validate_config`
501 $\rightarrow$ `run_simulation`.

502 Lazy retrieval, by contrast, is exercised only on failure recovery: it allows
503 the agent to query the KG *after* a simulation fails. Since budget-exhaustion
504 tasks never reach `run_simulation` at all, lazy retrieval cannot help them.
505 We therefore conclude that **warm-start injection is the dominant factor**
506 in KG Smart’s advantage over KG On: it directly prevents all 3 systematic
507 failures (100% of KG On’s remaining failures after auto-retry). Warm-start

also indirectly mitigates stochastic failures by reducing the iteration count (fewer decision points $\Rightarrow$ fewer opportunities for the LLM to produce a non-tool-call response). Lazy retrieval provides complementary value on simulation-error recovery but is secondary for the reliability gain.

7. Production Agent Quality Metrics

Beyond the controlled ablation, we analyse the system’s behaviour across all simulation runs accumulated during development, testing, and ablation experiments. Of these, 805 originate from an automated parameter sweep across 8 physics studies (geometry, material, boundary conditions, initial conditions, time-stepping, 3D geometry, Robin convection, and multi-material) run via `scripts/sweep_full_study.py`; the remainder are interactive and ablation runs accumulated through normal system use.

7.1. Success and Failure Modes

The 97.8% overall success rate (1,339/1,369 from PostgreSQL `simulation_runs`) indicates production-grade reliability for the full agent stack. Of the 2.2% failures, the majority are attributable to invalid boundary condition combinations (e.g. pure Neumann systems without a reference point), mesh resolution requests exceeding GPU memory, and numerical divergence on stiff transients.

7.2. Iteration Efficiency

Mean reasoning steps per completed task: Simulation Agent 11.1, Analytics Agent 10.2, Database Agent 6.2. The Simulation Agent’s count covers a full cycle of validating a configuration, revising it, launching the solver and checking the result, and configurations are often revised more than once

Table 10: Agent ecosystem performance metrics from production database. FEniCSx runs, through 2026-04-17 (excludes the controlled ablation campaign and non-FEniCSx backends).

Metric	Value
Total simulation runs	1369
Overall success rate	97.8%
Unique agent tasks	421
First-try success rate	85.4%
Retry rate	10.3%
Suggestion acceptance rate	0.0%
Avg. steps/task (analytics)	10.2
Avg. steps/task (database)	6.2
Avg. steps/task (simulation)	11.1
Avg. simulation time	2.49s

531 before launch. The Analytics Agent runs several comparison queries before
532 settling on a recommendation. The Database Agent’s lower count reflects
533 the relative simplicity of structured SQL-style queries over natural-language
534 reasoning.

535 7.3. *First-Try Rate*

536 85.4% of simulation tasks succeed on the first attempt, with no modification-
537 and-retry loop. Another 10.3% call the solver more than once, or fall back to
538 `debug_simulation`, before succeeding; these are usually boundary condition
539 corrections or CFL stability adjustments. The remaining 4.3% fail on a single
540 attempt and never retry.

541 Attempts are counted from `run_simulation` tool calls rather than from
542 distinct run identifiers. Counting identifiers does not work here: the agent
543 logger back-fills the most recent `run_id` across every step of a task once it is
544 known, which makes a task that retried look identical to one that did not.
545 Counting tool calls recovers the real attempt count. Retries turn out to be
546 less frequent than the aggregate success rate suggests, but they are still what
547 carries the system from 85.4% to 97.8% final success.

548 7.4. *KG Growth and Learning Curve*

549 To isolate the KG’s causal effect on performance, we run a *controlled*
550 growth experiment. The KG is initialised with only static material definitions
551 (13 materials, 45 reference entries) but *zero* Run nodes, so the agent has
552 no prior simulation experience. We then run 100 tasks (two shuffled passes
553 through the 50-task benchmark) sequentially in KG Smart mode with writes
554 enabled; each successful run adds one Run node with an HNSW-indexed

555 embedding and up to five **SIMILAR_TO** edges. By pass 2, the KG contains 50
 556 Run nodes from pass 1, providing warm-start context that was absent during
 557 pass 1.

558 To measure the effect, we perform a *paired comparison*: for each of the 50
 559 benchmark tasks, we compare success-only MPF and physics score between
 560 pass 1 (zero prior runs) and pass 2 (50 prior runs), retaining only the 33 non-
 561 novel tasks that succeeded in both passes. Novel tasks are excluded because
 562 their fidelity depends on pre-seeded Material nodes rather than accumulated
 563 Run history. fig. 9 shows the result stratified by difficulty.

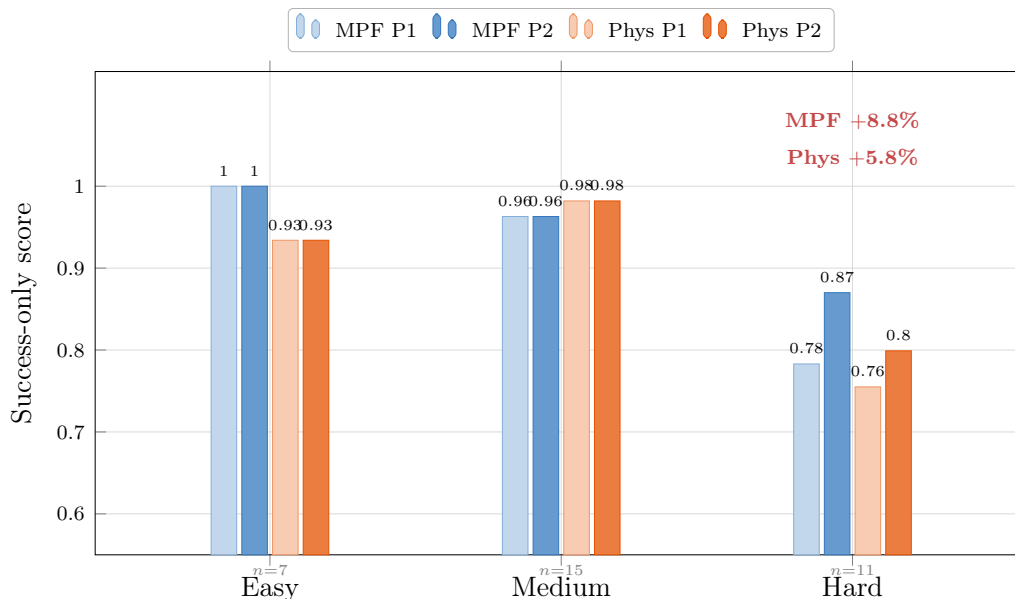


Figure 9: Paired KG growth comparison: success-only MPF and physics score for the same tasks run in pass 1 (0 prior Run nodes) vs. pass 2 (50 prior Run nodes). Only tasks succeeding in both passes are compared ($n=33$; novel tasks omitted as they rely on pre-seeded material definitions). Easy and medium tasks are at ceiling; **hard tasks gain +8.8% MPF and +5.8% physics score**, the category where similarity-based warm-start adds most value.

564 The experiment reveals a *stratified* growth effect. Easy and medium tasks
 565 already achieve $\text{MPF} \geq 0.96$ in pass 1 because their parameters are either
 566 given explicitly or well-known to the LLM; accumulated run history cannot
 567 improve what is already near-perfect. Hard tasks, however, involve ambiguous
 568 descriptions, mixed boundary conditions, and tricky numerics where the agent
 569 benefits most from seeing similar prior runs: MPF rises from 0.783 to 0.870
 570 (+8.8%), and physics score from 0.755 to 0.799 (+5.8%). The overall success
 571 rate is 88% (88/100), confirming that KG Smart remains robust even when
 572 starting without prior experience.

573 These results show that KG value is *difficulty-dependent*: static material
 574 knowledge covers simple cases, while accumulated run experience provides
 575 measurable gains on the hardest tasks where warm-start context disambiguates
 576 underspecified problems.

577 7.5. Comparison with Prior Work

578 *FEM code generation..* Bauer et al. [16] report $\approx 60\%$ one-shot success for
 579 GPT-4-driven FEniCS script generation. Our 85.4% first-try rate is higher, but
 580 the two numbers measure different tasks and are not directly comparable: their
 581 system has to emit a complete, syntactically valid FEniCS script, while ours
 582 fills in a parameter set for a fixed, pre-verified solver. What does carry across
 583 is the architecture. Unlike their single-turn approach, PDE-Agents recovers
 584 from failures through multi-turn self-correction, reaching 97.8% final success
 585 with a fully local, open-source model stack. ALL-FEM [18] addresses the
 586 FEniCS automation problem through fine-tuning: a corpus of 1,000+ verified
 587 FEniCS scripts is used to fine-tune models from 3B to 120B parameters, and
 588 a multi-agent framework orchestrates problem formulation, code generation,

589 and debugging. Their best model (GPT OSS 120B) achieves 71.8% code-
590 level success on 39 benchmarks spanning elasticity, plasticity, fluid flow, and
591 multiphysics, a substantially broader PDE scope than ours. PDE-Agents
592 differs in three respects: (i) we use unmodified open-source LLMs without
593 domain-specific fine-tuning, relying instead on tool-call orchestration and KG-
594 augmented prompting; (ii) our system manages the full simulation lifecycle
595 (configuration, execution, monitoring, debugging) rather than generating
596 standalone scripts; and (iii) we provide a formal V&V study and a controlled
597 ablation of knowledge graph integration modes, establishing when and why
598 retrieval augmentation helps.

599 *FEA benchmarks..* FEABench [19] provides a systematic evaluation of LLM
600 and agent capabilities for finite element analysis using COMSOL Multiphysics.
601 Their best agent strategy generates executable API calls 88% of the time
602 across problems in heat transfer, structural mechanics, and electromagnetics.
603 Our work complements FEABench by targeting the open-source FEniCSx
604 ecosystem and measuring not just execution success but *output quality* (physics
605 score, material property fidelity), exposing cases where computationally
606 “successful” simulations produce physically incorrect results due to fabricated
607 material properties.

608 *CFD agent systems..* The computational fluid dynamics community has been
609 particularly active in LLM-driven automation. OpenFOAMGPT 2.0 [38]
610 achieves 100% success across 450+ simulations using a four-agent pipeline
611 (pre-processing, prompt generation, simulation, post-processing) with RAG-
612 augmented retrieval. MetaOpenFOAM [39] decomposes CFD workflows into
613 subtasks using MetaGPT’s [40] assembly-line paradigm, reporting 85% pass

614 rates at \$0.22 per case. These systems validate the multi-agent pattern we
615 adopt but target a different solver ecosystem and do not evaluate knowledge
616 graph integration or measure output fidelity against ground-truth material
617 properties.

618 *KG-augmented agents for materials science..* Buehler [30] demonstrates
619 retrieval-augmented ontological knowledge graph strategies for materials
620 design with a fine-tuned MechGPT model, showing that graph-structured re-
621 trieval outperforms flat RAG by providing mechanistic relationships between
622 concepts. Our KG Smart pattern shares the same insight (that structured
623 knowledge improves over naïve retrieval) but applies it in a *live simulation*
624 *loop* rather than a generative design setting, and our ablation study provides
625 controlled evidence for when graph-augmented retrieval helps (novel materials)
626 versus when it adds overhead without benefit (standard tasks with well-known
627 properties).

628 8. System Limitations and Future Work

629 *Solver scope..* The current solver supports only scalar heat equations. This
630 is the main limitation of the present work. Extending to vector mechanics
631 (linear elasticity, Navier–Stokes) is not simply a matter of richer tool schemas;
632 it requires new solver kernels, each with its own variational form, function
633 spaces and verification.

634 The narrow scope is also what makes our central measurement possible.
635 Because the agent fills in parameters for a fixed, independently verified solver
636 instead of emitting solver code, every run produces a configuration whose
637 material properties can be checked directly against ground truth, which is

638 what MPF (section 6.4) measures. A system that generates solver code covers
639 far broader physics but cannot isolate property fidelity the same way, since a
640 failure may lie in the code, in the parameters, or in both.

641 The obvious next step is to test on physics we did not write ourselves.
642 ALL-FEM [18] has released a public benchmark of FEM problems with ground-
643 truth solutions covering elasticity, plasticity, Newtonian and non-Newtonian
644 flow, and fluid–structure interaction.¹ Most of these are beyond our solver
645 today, so evaluating on them means implementing the corresponding kernels
646 first. We see that work, combining KG-augmented memory with multi-physics
647 breadth, as more valuable than an incremental extension of the present study.

648 *Model scale and open-source evolution..* The system defaults have been up-
649 graded from the initial v1 models (qwen2.5-coder:32b, llama3.3:70b) to
650 Qwen3-Coder-Next (80B total, 3B active MoE) [41] and Llama 4 Scout
651 (109B total, 17B active MoE) [42], with cross-model validation confirming
652 model-agnostic KG Smart performance (table 8). Qwen3-Coder-Next achieves
653 higher config quality on standard KG Smart tasks (0.94 vs. 0.86) and perfect
654 MPF on novel tasks, identical to the v1 model. Llama 4 Scout provides
655 10M-token context and native tool calling for the orchestrator and analytics
656 agents. A systematic multi-model ablation across additional families (GLM-5,
657 DeepSeek-R2) is planned as future work.

658 *KG integration..* The KG Smart pattern (section 6.3) achieves 100% success
659 while producing the highest quality outputs, matching KG Off’s reliability.
660 KG On retains a small gap (94%) due to budget exhaustion. The adaptive

¹https://github.com/fenics-llm/FEM_Dataset

661 decision framework (Algorithm 1) provides practical deployment guidance,
662 achieving 100% mode-selection accuracy. Future work will explore: (a) rein-
663 forcement learning from simulation outcomes to tune KG retrieval relevance,
664 (b) dynamic confidence-based retrieval (only query KG when the LLM’s un-
665 certainty exceeds a threshold), and (c) richer warm-start context including
666 failure diagnostics from similar past runs.

667 *Is retrieval necessary, or would a better prompt suffice?* A fair objection to
668 any retrieval-augmented result is that the same information could simply be
669 written into the prompt. We tested this with a *prompt-injection* control on the
670 preliminary 7-task novel-material benchmark: the KG was disabled and the
671 true properties of Novidium, Cryonite and Pyrathane were appended verbatim
672 to each task description. It matched KG Smart on fidelity (MPF = 1.00) and
673 was the fastest mode overall (11.7s mean). That is the expected outcome,
674 since the answer was handed to the model.

675 The control still tells us something useful about where the difficulty lies.
676 Prompt injection assumes you already know which materials a task will
677 involve and what their properties are. It is a manual step, repeated per task,
678 and it does not scale beyond a handful of known entities. KG Smart reaches
679 the same fidelity by retrieving those properties through embedding similarity,
680 without being told which material the task names. The result worth reporting
681 is not that retrieved context helps, which is unsurprising, but that automatic
682 retrieval matches hand-curated context with no human in the loop.

683 *Verification scope.* The V&V study covers only steady-state and simple
684 transient heat equation benchmarks. A complete V&V programme would

685 include higher-order elements, singularity-containing domains, and time-
686 dependent convergence rates.

687 *Parallelism and scalability..* The current architecture runs agents sequentially.
688 A task-level parallelism layer (e.g. running a parametric sweep as concurrent
689 simulation agent calls) is under development.

690 *Knowledge graph evolution..* The 805-run parameter sweep has already created
691 a dense graph with 4,000+ **SIMILAR_TO** edges. Future work will mine this
692 corpus for **IMPROVED_OVER** relationships (did increasing mesh resolution reduce
693 error without proportionally increasing wall time?), enabling the agent to
694 autonomously suggest Pareto-optimal configurations. A second planned
695 enhancement is citation traceability: agents citing specific **ReferenceChunk**
696 IDs from indexed literature when justifying parameter choices, providing a
697 fully auditable reasoning trail.

698 9. Conclusion

699 We have presented PDE-Agents, an open, containerised multi-agent ecosys-
700 tem that automates finite element simulations through natural-language inter-
701 action with locally-deployed open-source LLMs. The system achieves 97.8%
702 overall success across 1,369 production runs and demonstrates $\mathcal{O}(h^2)$ spatial
703 convergence validated against analytical solutions.

704 *The KG Smart integration pattern..* Our three-way ablation study across 50
705 benchmark tasks with a frozen KG reveals that the *integration pattern*, not the
706 knowledge source itself, determines KG utility. KG Off (no knowledge graph)
707 achieves 100% task completion but fabricates material properties, producing

MPF = 0.34 and $\text{MPF}_w=0.21$ on novel materials. KG On (mandatory KG access) retrieves correct properties but retains a 6% failure rate from budget exhaustion and timeout (even after implementing agent-level retry to eliminate stochastic failures). KG Smart, combining warm-start injection with lazy conditional retrieval, achieves **100% success with the highest output quality** (physics 0.933, MPF 0.926), resolving the reliability-vs-fidelity trade-off.

Failure analysis: why warm-start dominates.. After implementing auto-retry to eliminate stochastic LLM failures, KG On’s 3 remaining failures (section 6.5) are all systematic: 2 budget-exhaustion cases and 1 timeout, all caused by the mandatory pre-simulation KG query loop. Warm-start injection eliminates this bottleneck entirely by providing material properties and reference configurations before the agent loop begins, accounting for KG Smart’s 6-pp success rate advantage. Lazy retrieval provides complementary value on failure recovery but is secondary for the reliability gain.

KG value is difficulty- and domain-dependent.. A controlled 100-task KG growth experiment shows that the KG’s benefit is concentrated where it matters most. Easy and medium tasks are already at quality ceiling ($\text{MPF} \geq 0.96$) from the first pass; hard tasks, involving ambiguous descriptions and mixed boundary conditions, gain +8.8% MPF and +5.8% physics score from accumulated run history. For novel materials, the KG is indispensable: fabricated properties cause output errors up to 949 K, while KG Smart eliminates these errors entirely.

731 *Lessons learned..* Three design principles emerged from the ablation and
732 failure analysis:

733 1. **Never make KG access mandatory in the agent’s critical path.**

734 Mandatory pre-simulation retrieval creates fragile coupling between
735 knowledge access and task completion. Front-loading context via em-
736 bedding similarity (warm-start) decouples them.

737 2. **Front-load context, don’t force the agent to query for it.** The

738 agent’s iteration budget is its scarcest resource. Injecting relevant
739 context into the system prompt before the agent loop is strictly more
740 efficient than requiring the agent to spend iterations formulating and
741 executing KG queries.

742 3. **Let the agent decide when it needs more knowledge.** Lazy

743 conditional retrieval respects the agent’s autonomy: it queries the KG
744 only after a failure or when material properties are genuinely unknown,
745 avoiding unnecessary round-trips on tasks where the LLM’s parametric
746 knowledge suffices.

747 *Broader implications..* The warm-start + lazy-retrieval pattern is not specific
748 to heat transfer simulation. Any tool-using agent domain with a growing
749 knowledge base (materials science, drug discovery, circuit design, structural
750 engineering) could benefit from the same architecture: embed the task, retrieve
751 the most similar prior successes, inject them as few-shot context, and defer
752 explicit knowledge queries to the failure-recovery path. As knowledge graphs
753 grow and LLMs improve, the quality gap between KG-enabled and KG-free
754 agents will only widen for novel or proprietary domains where the LLM’s
755 training data provides no coverage.

756 The production metrics confirm that multi-turn self-correction is essential:
757 the 85.4% first-try rate improves to 97.8% through iterative debugging, under-
758 scoring the value of agent-level orchestration over single-turn code generation.
759 PDE-Agents establishes both a working platform and a rigorous empirical
760 baseline for the emerging field of LLM-driven autonomous simulation, showing
761 that structured knowledge integration turns an agent from an “expensive
762 random number generator” into a reliable engineering tool.

763 **Data Availability**

764 All code, evaluation scripts, and result JSON files are available at <https://github.com/MatPro-IFE/pde-agents>. The Neo4j knowledge graph seed
765 data is included in the repository at `knowledge_graph/seeder.py`.
766

767 **CRedit authorship contribution statement**

768 **Sayan Adhikari:** Conceptualization, Methodology, Investigation, Soft-
769 ware, Data Curation, Formal Analysis, Visualization, Writing – original draft,
770 Writing – review & editing. **Gulshan Noorsumar:** Conceptualization,
771 Methodology, Investigation, Validation, Writing – review & editing. **Øyvind**
772 **Jensen:** Supervision, Writing – review & editing.

773 **Declaration of generative AI and AI-assisted technologies in the** 774 **manuscript preparation process**

775 During the preparation of this work the authors used Anthropic’s Claude
776 Opus 4.6 in order to assist with language editing, grammar refinement, and
777 the programmatic generation of TikZ figures (Figures 1 and 2). After using

778 this tool, the authors reviewed and edited the content as needed and take full
779 responsibility for the content of the published article.

780 **Declaration of competing interest**

781 The authors declare that they have no known competing financial interests
782 or personal relationships that could have appeared to influence the work
783 reported in this paper.

784 **Acknowledgements**

785 The authors thank the anonymous reviewers for their constructive feedback.
786 Computational resources were provided by the Institute for Energy Technology
787 (IFE).

788 **References**

- 789 [1] T. B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot
790 learners, *Advances in Neural Information Processing Systems* 33 (2020)
791 1877–1901.
- 792 [2] J. Wei, X. Wang, D. Schuurmans, et al., Chain-of-thought prompting
793 elicits reasoning in large language models, *Advances in Neural Information*
794 *Processing Systems* 35 (2022).
- 795 [3] S. Yao, J. Zhao, D. Yu, et al., ReAct: Synergizing reasoning and acting
796 in language models, *Proceedings of the International Conference on*
797 *Learning Representations (ICLR)* (2023).

- 798 [4] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for
799 solving ordinary and partial differential equations, IEEE Transactions
800 on Neural Networks 9 (5) (1998) 987–1000.
- 801 [5] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural
802 networks: A deep learning framework for solving forward and inverse
803 problems involving nonlinear partial differential equations, Journal of
804 Computational Physics 378 (2019) 686–707. doi:10.1016/j.jcp.2018.
805 10.045.
- 806 [6] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, et al., Physics-informed
807 machine learning, Nature Reviews Physics 3 (6) (2021) 422–440.
- 808 [7] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for
809 parametric partial differential equations, Proceedings of the International
810 Conference on Learning Representations (ICLR) (2021).
- 811 [8] J. Jumper, R. Evans, A. Pritzel, et al., Highly accurate protein structure
812 prediction with AlphaFold, Nature 596 (7873) (2021) 583–589. doi:
813 10.1038/s41586-021-03819-2.
- 814 [9] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation
815 for knowledge-intensive NLP tasks, Advances in Neural Information
816 Processing Systems 33 (2020) 9459–9474.
- 817 [10] D. Edge, H. Trinh, N. Cheng, et al., From local to global: A
818 graph RAG approach to query-focused summarization, arXiv preprint
819 arXiv:2404.16130 (2024).

- [11] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4) (2020) 824–836. [doi:10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [12] A. M. Bran, S. Cox, O. Schilter, et al., ChemCrow: Augmenting large-language models with chemistry tools, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2023.
- [13] Y. Ma, Z. Liu, L. P. Liang, et al., SciAgent: Tool-augmented language models for scientific reasoning, *arXiv preprint arXiv:2402.11451* (2024).
- [14] A. Logg, K.-A. Mardal, G. N. Wells, et al., *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*, Springer, 2012. [doi:10.1007/978-3-642-23099-8](https://doi.org/10.1007/978-3-642-23099-8).
- [15] I. A. Barrata, J. P. Dean, J. S. Dokken, et al., DOLFINx: The next generation FEniCS problem solving environment, *Zenodo* (2023). [doi:10.5281/zenodo.10447666](https://doi.org/10.5281/zenodo.10447666).
- [16] P. Bauer, P. Henning, J. Schaefer, Large language models as automatic generators of FEniCS code for solving partial differential equations, *arXiv preprint arXiv:2312.09801* (2023).
- [17] W. Jiang, K. Chen, M. Wang, et al., LLM4FEM: Leveraging large language models for finite element method, *arXiv preprint arXiv:2405.03719* (2024).
- [18] R. Deotale, A. Srinivasan, M. Golestanian, Y. Tian, T. Zhang, P. Vlachos, H. Gomez, ALL-FEM: Agentic large language models fine-tuned for

finite element methods, Computer Methods in Applied Mechanics and Engineering 457 (2026) 118985. doi:10.1016/j.cma.2026.118985.

[19] N. Mudur, H. Cui, S. Venugopalan, P. Raccuglia, M. P. Brenner, P. Norgaard, FEABench: Evaluating language models on multiphysics reasoning ability, arXiv preprint arXiv:2504.06260 (2025).

[20] LangChain AI, [LangGraph: Build stateful, multi-actor applications with LLMs](#) (2024).
URL <https://github.com/langchain-ai/langgraph>

[21] Q. Wu, G. Bansal, J. Zhang, et al., AutoGen: Enabling next-generation LLM applications via multi-agent conversation, in: Proceedings of EMNLP Industry Track, 2023.

[22] J. Moura, [CrewAI: Framework for orchestrating role-playing, autonomous AI agents](#) (2024).
URL <https://github.com/joaomdmoura/crewai>

[23] X. Liao, H. Zhang, Y. Chen, Retrieval-augmented generation for engineering design documentation, arXiv preprint arXiv:2307.04512 (2023).

[24] Y. Gao, S. Liu, P. Shi, J. Lin, Retrieval-augmented code generation for universal information extraction, arXiv preprint arXiv:2311.02555 (2023).

[25] Z. Yang, W. Li, P. Zhang, Simulation parameter suggestion via retrieval-augmented generation, arXiv preprint arXiv:2403.09512 (2024).

- [26] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024, arXiv:2401.15884.
- [27] P. Anokhin, N. Kornaev, A. Babkin, A. I. Panov, AriGraph: Learning knowledge graph world models with episodic memory for LLM agents, in: Advances in Neural Information Processing Systems (NeurIPS), 2024, arXiv:2407.04363.
- [28] V. Venugopal, S. Sahoo, G. Agastya, et al., MatKG: The largest knowledge graph in applied materials science, arXiv preprint arXiv:2209.11632 (2022).
- [29] C. W. Andersen, R. Armiento, E. Blokhin, et al., OPTIMADE: Towards an open database for computational materials science, Scientific Data 8 (1) (2021) 217. [doi:10.1038/s41597-021-00974-z](https://doi.org/10.1038/s41597-021-00974-z).
- [30] M. J. Buehler, Generative retrieval-augmented ontologic graph and multiagent strategies for interpretive large language model-based materials design, ACS Engineering Au 4 (2) (2024) 241–277. [doi:10.1021/acsengineeringau.3c00058](https://doi.org/10.1021/acsengineeringau.3c00058).
- [31] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities, International Journal for Numerical Methods in Engineering 79 (11) (2009) 1309–1331. [doi:10.1002/nme.2579](https://doi.org/10.1002/nme.2579).
- [32] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, A. Taylor, Cypher: An evolving

- 887 query language for property graphs, in: Proceedings of the 2018 In-
 888 ternational Conference on Management of Data (SIGMOD), 2018, pp.
 889 1433–1445. doi:10.1145/3183713.3190657.
- 890 [33] Z. Nussbaum, J. X. Morris, B. Duderstadt, A. Mulyar, Nomic embed:
 891 Training a reproducible long context text embedder, arXiv preprint
 892 arXiv:2402.01613 (2024).
- 893 [34] IBM Research, Docling: Document processing for AI (2024).
 894 URL <https://github.com/DS4SD/docling>
- 895 [35] ASME, Guide for verification and validation in computational solid me-
 896 chanics, Tech. Rep. ASME V&V 10-2006, American Society of Mechanical
 897 Engineers (2006).
- 898 [36] E. B. Wilson, Probable inference, the law of succession, and statistical
 899 inference, Journal of the American Statistical Association 22 (158) (1927)
 900 209–212. doi:10.1080/01621459.1927.10502953.
- 901 [37] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd
 902 Edition, Lawrence Erlbaum Associates, 1988.
- 903 [38] H. Chen, L. Mangani, G. Casas, OpenFOAMGPT 2.0: End-to-end,
 904 trustworthy automation for computational fluid dynamics, arXiv preprint
 905 arXiv:2504.19338 (2025).
- 906 [39] Y. Chen, X. Zuo, Y. Yang, et al., MetaOpenFOAM: An LLM-based
 907 multi-agent framework for CFD, arXiv preprint arXiv:2407.21320 (2024).

- [40] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, MetaGPT: Meta programming for a multi-agent collaborative framework, arXiv preprint arXiv:2308.00352 (2024).
- [41] R. Cao, M. Chen, J. Chen, Z. Cui, Y. Feng, B. Hui, Y. Jing, K. Li, M. Li, J. Lin, Z. Ma, K. Shum, X. Wang, J. Wei, J. Yang, J. Zhang, L. Zhang, Z. Zhang, W. Zhao, F. Zhou, Qwen3-coder-next technical report, arXiv preprint arXiv:2603.00729 (2026).
- [42] Meta AI, The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation, <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, accessed 2026-04-15 (2025).

Appendix A. Full h-Refinement Data

Table A.11 reports the full mesh-refinement study behind the convergence rates summarised in table 2: L^2 and L^∞ error norms at every resolution $N \in \{8, 16, 32, 64, 128\}$ for all three benchmark cases.

The two norms are computed differently, and for one case they disagree by several orders of magnitude. $\|e\|_{L^2}$ is assembled by quadrature over each element, so it measures error throughout the domain, including between nodes. $\|e\|_{L^\infty}$ is evaluated only at the degrees of freedom. For the constant-source (1D-like Poisson) case the P1 solution is *nodally exact*, which is a standard superconvergence result, so the nodal norm sits at machine precision ($\approx 10^{-10}$) at every resolution while the integrated norm decays at $\mathcal{O}(h^2)$ as expected. Convergence rates are therefore fitted from $\|e\|_{L^2}$ alone; a rate fitted from

⁹³¹ nodal values would say nothing useful about this case.

Table A.11: Detailed convergence data: error norms at each mesh resolution. $\|e\|_{L^2}$ is integrated over each element with quadrature of degree 8, so it captures the error *between* nodes; $\|e\|_{L^\infty}$ is evaluated at the degrees of freedom only. For the constant-source case the P1 solution is nodally exact, so $\|e\|_{L^\infty}$ sits at machine precision while $\|e\|_{L^2}$ shows the expected $\mathcal{O}(h^2)$ interpolation error. The two norms measure different things, and only the integrated norm is used to fit convergence rates.

Case	N	DOFs	$\ e\ _{L^2}$	$\ e\ _{L^\infty}$
2D Steady-State Linear Profile	8	81	4.22e-11	1.12e-10
	16	289	3.65e-09	1.44e-08
	32	1,089	3.89e-10	1.11e-09
	64	4,225	1.10e-09	3.00e-09
	128	16,641	2.47e-09	6.40e-09
2D Transient Fourier Mode Decay	8	81	4.38e-03	4.88e-03
	16	289	9.48e-04	2.64e-03
	32	1,089	2.39e-04	6.62e-04
	64	4,225	5.98e-05	1.65e-04
	128	16,641	1.50e-05	4.13e-05
2D Steady-State with Constant Source (1D-like)	8	81	1.43e-03	1.12e-10
	16	289	3.57e-04	2.28e-11
	32	1,089	8.91e-05	1.22e-10
	64	4,225	2.23e-05	1.83e-10
	128	16,641	5.57e-06	2.68e-10